

E-commerce SQL Roadmap

130 Business SQL Problems & Solutions

PostgreSQL | Data Analytics | Business Reporting

Prepared by DG

Disclaimer

This document is a personal learning portfolio created for educational purposes. It is based on an anonymized e-commerce dataset. Any company names, customer information, vendor names, employee details, order identifiers, and other sensitive information have been removed, anonymized, or replaced with fictional values. Any resemblance to real organizations, customers, or vendors is purely coincidental.

Table of Contents

Chapter 1 – SQL Basics (20 Questions)	2
Topics: SELECT, DISTINCT, WHERE, ORDER BY, LIMIT, Aliases	2
Chapter 2 – Aggregate Functions (20 Questions)	7
Topics: COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING.....	7
Chapter 3 – SQL Functions (16 Questions)	11
Topics: String, Date, Numeric, CASE, COALESCE	11
Chapter 4 – JOINS (10 Questions).....	13
Chapter 5 – Advanced Subqueries & CTEs (8 Questions)	17
Chapter 6 – Window Functions (18 Questions)	20
Chapter 7 – Advanced SQL (9 Questions)	26
Chapter 8 - Final Business Challenge (29 Questions)	30
Key Learning Outcomes :	37
Skills Demonstrated	37

Chapter 1 – SQL Basics (20 Questions)

Topics: SELECT, DISTINCT, WHERE, ORDER BY, LIMIT, Aliases

1. Show all records from order_report.

Business Understanding:

(What is the question asking?) : To show all the record of the order table

SQL Query:

```
Select * from order_report;
```

2. Display only Order ID, Order Date and Customer ID.

SQL Query:

```
Select OrderID, OrderDate , CustomerID
```

```
from order_report;
```

3. Find all orders from a specific state.

SQL Query:

```
select distinct orderid, ShipToState
```

```
from order_report
```

```
where ShipToState in ('Alaska' , 'Hawaii');
```

4. Find all orders with Order Status = 'Shipped'.

SQL Query:

```
select orderid, ItemStatus
```

```
from order_report where ItemStatus = 'Shipped';
```

Explanation:

(Giving order status item status wise)

5. Show products having Qty greater than 5.

Business Understanding:

more than 5 will show for each line items

SQL Query:

```
select ItemDesc, Qty
```

```
from order_report
```

```
where qty>5;
```

6. Display unique payment types.

Business Understanding:
(What is the question asking?)

SQL Query:

```
select distinct PaymentType  
from order_report;
```

7. Display unique product categories.

Business Understanding:
to see the product category in business

SQL Query:

```
select distinct categorymain  
from order_report
```

8. Find all tax exempt orders.

SQL Query:

```
select distinct orderid, shiptostate, saletax  
from order_report  
where saletax = 0;
```

business understanding: Sales tax of 0 represents tax-exempt orders where no sales tax is charged to the customer

9. Find all orders with coupon applied.

SQL Query:

```
select distinct orderid, CouponCode  
from order_report  
where CouponCode is not null;
```

10. Find all AutoShip orders.

SQL Query:

```
select distinct orderid  
  
from order_report  
  
where CouponCode = 'AutoShipDiscount' or AutoShipFrequency > 0;
```

business understanding:

An order is identified as an AutoShip order if an AutoShip discount is applied or the AutoShip frequency is greater than zero.

11. Show top 20 highest amount orders.

SQL Query:

```
select orderid, sum(Amount) as total_amount  
  
from order_report  
  
group by orderid  
  
order by total_amount desc limit 20;
```

Explanation:

(using aggregate fn in order by clause gives the total order with sum of their line items.)

12. Show lowest 20 amount orders.

```
select orderid, sum(Amount) as total_amount  
  
from order_report  
  
group by orderid  
  
order by total_amount asc limit 20;
```

13. Find orders between two dates.

SQL Query:

```
select distinct orderid  
  
from order_report  
  
where orderdate between '6/1/2026' and '6/07/2026'
```

14. Find orders from a specific customer.(tiers wise)

SQL Query:

```
select loyaltyname, count(distinct orderid) as unique_order_count
from order_report
group by loyaltyname
order by unique_order_count desc;
```

15. Find purchase orders from a specific vendor.

SQL Query:

```
select distinct vendorname, sum(totalcostprice)
from purchase_order
group by vendorname
having vendorname = 'samson'
```

16. Find purchase orders added by a specific employee.

SQL Query:

```
select
coalesce (poaddedby,'Grand_Total')
poaddedby, count(distinct porderno)
from purchase_order
group by rollup (poaddedby);
```

17. Show all red flag purchase orders item status wise .

SQL Query:

```
select
coalesce (orderitemstatus,'Grand_total'),
count (distinct porderno), sum(totalcostprice)
from purchase_order
```

group by rollup (orderitemstatus)

18. Display only selected columns with aliases.

SQL Query:

```
select distinct porderno, orderitemstatus from purchase_order
where orderitemstatus = 'Cancelled';
```

19. Sort customers by total amount descending.

SQL Query:

```
select CustomerId,
concat(BillFirstName,' ', BillLastName) as full_name,
sum(Amount) as total_amount_spent
from order_report
group by CustomerId, BillFirstName, BillLastName
order by total_amount_spent desc;
```

20. Display first 50 records.

SQL Query:

```
SELECT *
FROM order_report
ORDER BY orderdate
LIMIT 50;
```

Chapter 2 – Aggregate Functions (20 Questions)

Topics: COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING

1. Total number of orders.

```
select distinct OrderId
from order_report
```

2. Total revenue.

```
select sum(amount) as total_revenue  
from order_report
```

3. Average order value.

```
with avg_amount as(  
select sum(amount) as total_revenue  
from order_report  
group by orderid  
)
```

```
select avg(total_revenue) as avg_revenue  
from avg_amount;
```

4. Highest order amount.

```
select distinct orderid, sum(amount) as total_amount  
from order_report  
group by orderid  
order by total_amount desc limit 1;
```

5. Lowest order amount

```
select distinct orderid, sum(amount) as total_amount  
from order_report  
group by orderid  
order by total_amount asc limit 1;
```

6. Total quantity sold.

```
select sum(Qty)  
from order_report
```

7. Orders by status.

```
select orderstatus, count(distinct orderId)
```



```
from order_report  
group by orderstatus;
```

8. Revenue by state.

```
select ShipToState, sum(Amount)  
from order_report  
group by ShipToState order by ShipToState;
```

9. Revenue by category.

```
select category, sum(Amount)  
from order_report  
group by category order by category asc;
```

10. Revenue by payment type.

```
select PaymentType, sum(Amount)  
from order_report  
group by PaymentType order by PaymentType asc;
```

11. Revenue by affiliate

```
select AffiliateOrderNo, sum(Amount)  
from order_report  
group by AffiliateOrderNo  
having AffiliateOrderNo is not null;
```

12. Revenue by store.

```
select StoreName, sum(Amount)  
from order_report  
group by StoreName order by StoreName asc;
```

13. Top 10 customers by revenue.

```
select CustomerId,  
BillFirstName || ' ' || BillLastName as full_name,  
sum(Amount) as total_amount_spend  
from order_report
```

group by CustomerId, full_name order by total_amount_spend desc limit 1;

14. Top 10 products by sales.

```
select productcode, productname, sum(totalcostprice) as
total_revenue_byproduct

from purchase_order

group by productcode, productname

order by total_revenue_byproduct desc limit 10;
```

15. Vendor-wise purchase cost.

```
select vendorname, sum(totalcostprice) as totalcostprice

from purchase_order

group by vendorname order by totalcostprice desc;
```

16. Average selling price by product.

```
select productcode, round(avg(totalcostprice),2) as avg_sales

from purchase_order

group by productcode
```

17. Total shipping amount collected.

A. Ans with cte:

```
with true_shipping as (

select distinct orderid, max(ShippingAmount) as true_shipping

from order_report

group by orderid

)

select sum(true_shipping) as total_shipping_collected

from true_shipping;
```

B. ans with subqueries:

```
SELECT SUM(ShippingAmount) AS total_distinct_shipping

FROM (

SELECT DISTINCT ON (OrderId) OrderId, ShippingAmount
```

```
FROM order_report  
ORDER BY OrderId  
) unique_orders;
```

18. Total sales tax collected.

```
select sum(SalesTax)  
from order_report;
```

19. Categories having sales greater than X.

```
select Category, sum(Amount) as total_revenue  
from order_report  
group by Category  
having sum(Amount) > 50000
```

20. Vendors with purchase cost above X

```
select vendorname, sum(totalcostprice) as vendor_purchase_cost  
from purchase_order  
group by vendorname  
having sum(totalcostprice) > 20000;
```

Chapter 3 – SQL Functions (16 Questions)

Topics: String, Date, Numeric, CASE, COALESCE

1. Convert customer names to uppercase.

```
select concat ( upper (ShipFirstName), ' ', upper (ShipLastName))  
from order_report
```

2. Display full customer name.

```
select concat ( upper (BillFirstName), ' ', upper (BillLastName)) as full_name  
from order_report;
```

3. Find length of product names.

```
select ItemDesc, length (ItemDesc) as length_of_product
```

```
from order_report
```

4. Extract month from Order Date.

```
select OrderDate, extract (month from OrderDate) as months  
from order_report;
```

5. Extract year from Order Date.

```
select OrderDate, extract (year from OrderDate) as months  
from order_report;
```

6. Count orders by day

```
select orderdate as days, count(distinct orderid) as total_order_perday  
from order_report  
group by orderdate;
```

7. Round selling price.

```
select orderid, round (Amount,2) as rounded_sp  
from order_report;
```

8. Display coupon codes, replacing NULL values with 'No Coupon

```
select distinct <orderid, CouponCode,  
coalesce(CouponCode,'Not_avail') as couponstatus  
from order_report
```

9. Classify High/Medium/Low value orders using CASE.

```
select orderid, sum(amount) as total_amount,  
case  
    when sum(amount) >= 200 then 'High_value'  
    when sum(amount) >= 50 then 'Medium_value'  
    else 'Low_value'  
end as amount_category  
from order_report  
group by orderid;
```

10. Display weekday of order.

```
select distinct orderid, to_char(orderdate, 'day') as weekday  
from order_report
```

11. Display month name.

```
select distinct orderid, to_char(orderdate, 'month') as weekday  
from order_report
```

12. Trim extra spaces.

```
select ItemDesc, trim(ItemDesc)  
from order_report
```

13. Display first 10 characters of Item Description.

```
select ItemDesc, left( ItemDesc,10)  
from order_report
```

14. Find orders placed on a specific date.

```
select orderid  
from order_report  
where orderdate = '2026-06-22'
```

15. Find last day of month sales.

```
select orderdate, sum(amount)  
from order_report  
where orderdate = '2026-06-28'  
group by orderdate;
```

16. Format dates with DD-MM-YYYY format

```
select distinct orderid, orderdate, to_char(orderdate, 'dd-mm-yyyy')  
from order_report order by orderdate;
```

Chapter 4 – JOINS (10 Questions)

1. Join Orders with Purchase Orders.

```
select o.orderid, p.orderno, o.itemdesc
from order_report o
join purchase_order p
on o.orderid = p.orderno;
```

2. Show vendor for each customer order.

```
select o.orderid, o.itemdesc, p.vendorname
from order_report o
join purchase_order p
on o.item = p.productcode
order by o.orderid;
```

3. Show purchase cost and selling amount.

```
select
o.orderid, o.item, o.amount, p.total_cost
from order_report o
left join (
select orderno, productcode, uom,
sum(totalcostprice) as total_cost
from purchase_order
group by orderno, productcode, uom
) as p
```

```
on o.orderid= p.orderno
and o.item = p.productcode
and o.uom = p.uom
order by o.orderid;
```

4. Orders without purchase orders.

```
select o.orderid
```

```

from order_report o
left join purchase_order p
on o.orderid = p.orderno
where p.orderno is null;

```

5. Purchase orders without customer orders.

```

select p.orderno
from purchase_order p
left join order_report o
on o.orderid = p.orderno
where o.orderid is null;

```

6. Vendor-wise sales

```

select p.vendorname,
       sum(o.amount) as total_sales
       from order_report o
join purchase_order p
on split_part(o.uniqueid,'-',1)::bigint = p.productid
group by p.vendorname;

```

7. Customer-wise purchase cost.

```

select o.orderid, o.amount,
       sum(p.totalcostprice) as purchase_cost
       from order_report o
join purchase_order p
on o.orderid = p.orderno
and o.item = p.productcode
and o.uom = p.uom
group by o.orderid, o.amount;

```

8. Order id and Product-wise cost vs selling

```

select o.orderid, o.item, o.itemdesc, o.amount,
       sum(p.totalcostprice) as purchase_cost
from order_report o
join purchase_order p
on o.orderid = p.orderno
and o.item = p.productcode
and o.uom = p.uom
group by o.orderid, o.amount, o.item, o.itemdesc, o.amount;

```

9. Order fulfillment analysis

```

select o.orderid, p.shipmentdate
from order_report o
join purchase_order p
on o.orderid = p.orderno
where p.shipmentdate is not null;

```

10. Vendor performance by profit (%)age margin

```

(from order_report o
join (
select
    orderno, productcode, vendorname,
    sum(totalcostprice) as purchase_cost
from purchase_order
group by orderno, productcode, vendorname
) p
on o.orderid = p.orderno
and o.item = p.productcode
group by p.vendorname
order by pft_margin_prcntg desc;)

```

Chapter 5 – Advanced Subqueries & CTEs (8 Questions)

1. Customers above average spending.

```
with total_spending as
(
select customerid,
concat(billfirstname,' ',billlastname) as full_name,
sum(amount) as total_spent
from order_report
group by customerid, full_name)
select customerid, full_name, total_spent
from total_spending
where total_spent > (
select avg(total_spent) from total_spending
)
```

2. Products above average sales.

```
with sales_data as (
select item, itemdesc, uom, sum(amount) as total_sales
from Order_report
group by item, itemdesc, uom
order by total_sales desc
)
select item, itemdesc, uom, total_sales
from sales_data
where total_sales > ( select avg(total_sales) from sales_data);
select avg(amount) from Order_report
```

3. Highest selling category

```
select category, itemdesc, item, uom, sum(amount) as sales_data
from Order_report
group by category, itemdesc, item, uom
order by sales_data desc limit 1;
```

4. Vendors above average cost.

```
with total_costs as (
select vendorname, sum(totalcostprice) as total_cost_vendor
from purchase_order
group by vendorname)
select vendorname, total_cost_vendor
from total_costs
where total_cost_vendor > ( select avg(total_cost_vendor) from total_costs);
```

5. Monthly revenue CTE

```
WITH monthly_revenue AS (
    SELECT
        DATE_TRUNC('month', orderdate) AS months,
        SUM(amount) AS revenue
    FROM order_report
    GROUP BY DATE_TRUNC('month', orderdate)
)
SELECT
    months,
    revenue
FROM monthly_revenue;
```

6. Customer ranking CTE.

```

with customer_order_count as (
select customerid,
concat (billfirstname, ' ',billlastname) as full_name,
count(*) as total_orders
from order_report
group by customerid, full_name
)
select customerid, full_name, total_orders,
rank() over (order by total_orders desc) as customer_rank
from customer_order_count;

```

7. Repeat customers.

```

with repeate_cte as
(
select customerid,
concat (billfirstname, ' ',billlastname) as full_name,
count(*) as total_orders
from order_report
group by customerid, full_name
)
select customerid, full_name, total_orders
from repeate_cte
where total_orders>1;

```

8. Running business summary (running total of revenue)

```

select orderdate, sum(amount) as total_amount,
sum(sum(amount)) over ( order by orderdate ) as running_revenue
from order_report
group by orderdate
order by orderdate;

```

Chapter 6 – Window Functions (18 Questions)

1. Rank customers by revenue.

```
with customer_rank_cte as (  
  select customerid,  
  concat (billfirstname, ', ', billlastname) as full_name,  
  sum(amount) as total_spent  
  from order_report  
  group by customerid, full_name  
  order by total_spent desc  
)  
select customerid, full_name, total_spent,  
rank() over (order by total_spent desc) as customer_ranking  
from customer_rank_cte;
```

2. Rank products by sales.

```
with cte_product_rank as (  
  select item, itemdesc, uom, sum(amount) as total_sales  
  from order_report  
  group by item, itemdesc, uom  
  order by total_sales desc  
)  
select item, itemdesc, uom, total_sales,  
rank() over (order by total_sales desc) as sales_rank  
from cte_product_rank;
```

3. Dense Rank products

```
with cte_product_rnk as (  
  select item, itemdesc, uom,
```

```

count(*) as number_of_count
from order_report
group by item, itemdesc, uom)
select item, itemdesc, uom, number_of_count,
dense_rank() over (order by number_of_count desc) as product_rnk
from cte_product_rnk;

```

4. Row Number for orders.

```

select orderid, row_number() over (order by orderid asc ) as rownumber
from order_report;

```

5. Running revenue.

```

select orderdate, sum(amount) as total_amount,
sum(sum(amount)) over (order by orderdate) as running_total
from order_report
group by orderdate order by orderdate;

```

6. Running quantity.

```

select item, itemdesc, uom, sum(qty) as total_qty,
sum(sum(qty)) over (order by itemdesc) as running_total
from order_report
group by item, itemdesc, uom;

```

7. Moving average sales.

```

with moving_avg_cte as (
select orderdate, count(distinct orderId) as total_orders, sum(amount) as
daily_revenue
from order_report
group by orderdate
)
select orderdate, total_orders, daily_revenue,

```

```
round(avg(daily_revenue) over (order by orderdate rows between 6 preceding and
current row),2) as rolling_7_day_avg
```

```
from moving_avg_cte;
```

8. Previous order amount

```
with order_totals as (
```

```
select orderid, orderdate,
```

```
sum(amount) as current_order_amount
```

```
from order_report
```

```
group by orderid, orderdate order by ordered )
```

```
select orderid, orderdate, current_order_amount,
```

```
lag (current_order_amount) over (ORDER BY orderdate, orderid) as
previous_order_amount
```

```
from order_totals;
```

9. Next order amount

```
with total_amt as (
```

```
select orderid, orderdate, sum(amount) as total_order_amt
```

```
from order_report
```

```
group by orderid, orderdate)
```

```
select orderid, orderdate, total_order_amt,
```

```
lead (total_order_amt) over(order by orderdate, orderid) as next_order_amt
```

```
from total_amt;
```

10. Highest order per customer

```
select customerid,
```

```
concat(billfirstname,' ',billlastname) as full_name,
```

```
count( DISTINCT ORDERID) as total_count
```

```
from order_report
```

```
group by customerid, full_name ORDER BY total_count DESC; select
customerid,
```

```
concat(billfirstname,' ',billlastname) as full_name,
```

```

count( DISTINCT ORDERID) as total_count
from order_report
group by customerid, full_name ORDER BY total_count DESC;

```

11. WEEKLY running revenue.

```

with weekly_revnuue as (
select
    date_trunc('week',orderdate)::date as week_start,
    sum(amount) as weekly_revenue
FROM order_report
group by week_start
)
select week_start, weekly_revenue,
    sum(weekly_revenue) over (ORDER BY week_start) as running_rvnue
from weekly_revnuue;

```

12. Revenue contribution %.

```

select customerid,
    concat(billfirstname,' ',billlastname) as full_name,
    sum(amount) as customer_revenue,
    round(sum(amount) *100 / sum(sum(amount)) over(),2) as
revenue_contribution_pct
from order_report
group by customerid, full_name
order by revenue_contribution_pct desc;

```

13. Top 3 products per category.

```

with rank_cte as (
select categorymain, item, count(item) as top_products
from order_report
group by categorymain, item

```

```

),
ranked_product as (
select categorymain, item, top_products,
dense_rank () over ( partition by categorymain order by top_products desc) as
product_rank
from rank_cte
)
select categorymain, item, top_products, product_rank
from ranked_product where product_rank <= 3;
shot exp (even if the item count is 1 , its ranking will be 1 and will be considered
in it)

```

14. Vendor ranking.

```

with vendor_cte_rnk as (
select vendorname, sum (totalcostprice) as total_revenue
from purchase_order
group by vendorname
)
select *,
dense_rank() over (order by total_revenue desc) as vendor_rank
from vendor_cte_rnk;

```

15. State ranking by revenue

```

with state_rnk_cte as (
select shiptostate, sum(amount) as state_rvnu
from order_report
group by shiptostate
)
select *,
dense_rank() over( order by state_rvnu desc) as state_rnk

```



```
from state_rnk_cte;
```

16. Category ranking.

```
WITH category_cte AS (  
    SELECT  
        categorymain,  
        SUM(amount) AS revenue  
    FROM order_report  
    GROUP BY categorymain  
)  
SELECT *,  
DENSE_RANK() OVER (ORDER BY revenue DESC) AS category_rank  
FROM category_cte;
```

17. Customer lifetime ranking.

```
WITH customer_cte AS (  
    SELECT  
        customerid,  
        SUM(amount) AS lifetime_value  
    FROM order_report  
    GROUP BY customerid  
)  
SELECT *,  
DENSE_RANK() OVER (ORDER BY lifetime_value DESC) AS customer_rank  
FROM customer_cte;
```

18. Weekly trend analysis compared with previous week

```
with weekly_sales as (  
    select date_trunc('week',orderdate)::date as weekstart,  
    sum(amount) as weekly_rvnu  
    from order_report
```

```

group by date_trunc('week',orderdate)
),
diff as (
select *,
lag (weekly_rvnu) over(order by weekstart ) as previous_week_revnu
from weekly_sales
)
select *, weekly_rvnu - previous_week_revnu as diff_of_rev
from diff;

```

Chapter 7 – Advanced SQL (9 Questions)

1. Create sales view.

```

create view sales_view2 as

select

o.orderid,    o.customerid,    o.shiptocountry,    o.shiptotown,o.item,
            o.qty, o.price,    o.amount,    o.uom,    o.orderstatus,
            o.shippingamount, o.couponcode,    o.uniqueid,

p.porderno,  p.podate,    p.orderdate, p.orderno,    p.vendorname,
            p.totalcostprice,  p.poaddedby, p.productid, p.productcode,
            p.productname,    p.shipmentdate

FROM order_report o

left JOIN purchase_order p

    ON o.orderid = p.orderno

    AND o.item = p.productcode;

```

Interview notes:

- View stores the **query**, not the data.
- Changes in base tables are reflected automatically in the view.

- LEFT JOIN is preferred here because all customer orders should appear even if a purchase order is missing.
2. Build Reusable Reports Using Views


```
SELECT *
FROM sales_view2
WHERE orderdate = '6/2/2026';
Or
SELECT *
FROM sales_view
WHERE orderstatus <> 'Closed';
```
 3. Create Dashboard-Ready Queries
 - A.

```
SELECT
    item,
    SUM(qty) AS total_qty
FROM order_report
GROUP BY item
ORDER BY total_qty DESC
LIMIT 10;
```
 - B.

```
select shiptocountry, sum(amount) as total_sales
from order_report
group by shiptocountry
order by total_sales desc;
```
 4. Find Duplicate Records


```
SELECT
    orderid,
    item, UOM, itemdesc,
    COUNT(*) AS duplicate_count
FROM order_report
GROUP BY
    orderid,
    item, UOM, itemdesc
HAVING COUNT(*) > 1;
```
 5. Perform Data Quality Checks


```
SELECT * FROM order_report WHERE amount < 0;
OR
SELECT * FROM order_report WHERE qty <= 0;
```

OR

```
SELECT orderid, COUNT(*)  
FROM order_report GROUP BY orderid HAVING COUNT(*) > 1;
```

ANS:

- Check for **NULL values**.
- Check for **duplicate records**.
- Validate numeric values (e.g., qty > 0, amount >= 0).
- Validate dates (e.g., no future dates if business doesn't allow them).
- Verify relationships between tables using LEFT JOIN.

6. Create Indexes

```
<CREATE INDEX idx_orderstatus  
ON order_report(orderstatus);
```

```
select * from order_report  
where orderstatus = 'Cancelled'  
or
```

```
CREATE INDEX idx_orderdate  
ON order_report(orderdate);
```

```
select * from order_report  
where orderdate = '2026-06-15'
```

Composite index:

```
CREATE INDEX idx_customer_orderdate  
ON order_report(customerid, orderdate);
```

```
SELECT *  
FROM order_report  
WHERE customerid = 10000X  
AND orderdate BETWEEN '6/1/2026' AND '6/2/2026';
```

7. Compare Execution Plans (EXPLAIN & EXPLAIN ANALYZE)

Notes:

Objective

To understand **how PostgreSQL executes a query** and identify opportunities to improve its performance

Result

explain analyze

```
SELECT orderid, orderdate  
FROM order_report
```

```
WHERE customerid = 10000X
AND orderdate BETWEEN '6/1/2026' AND '6/2/2026';
```

Result:

```
"Index Scan using idx_customer_orderdate on order_report (cost=0.28..8.30
rows=1 width=12) (actual time=0.059..0.062 rows=2.00 loops=1)"
```

```
" Index Cond: ((customerid = 10000X) AND (orderdate >= '2026-06-01'::date)
AND (orderdate <= '2026-06-02'::date))"
```

```
" Index Searches: 1"
```

```
" Buffers: shared hit=3"
```

```
"Planning Time: 0.284 ms"
```

```
"Execution Time: 0.098 ms"
```

8. Optimize a Slow Query

Slow Query

```
SELECT * FROM order_report WHERE customerid = 10000X;
```

OPTIMIZED QUERY:

```
EXPLAIN ANALYZE
```

```
SELECT *
```

```
FROM order_report
```

```
WHERE customerid = 10000X
```

```
AND orderdate BETWEEN '2026-06-01' AND '2026-06-02';
```

Query plan:

```
"Index Scan using idx_customer_orderdate on order_report (cost=0.28..8.30
rows=1 width=12) (actual time=0.072..0.075 rows=2.00 loops=1)"
```

```
" Index Cond: ((customerid = 10000X) AND (orderdate >= '2026-06-01'::date)
AND (orderdate <= '2026-06-02'::date))"
```

```
" Index Searches: 1"
```

```
" Buffers: shared hit=3"
```

```
"Planning Time: 0.602 ms"
```

```
"Execution Time: 0.111 ms"
```

Interview tip

How do you optimize a slow SQL query?

"I first analyze the query using EXPLAIN ANALYZE to understand the execution plan. Then I check whether PostgreSQL is performing a Sequential Scan or using an Index Scan. If necessary, I create appropriate indexes, avoid applying functions to indexed columns in the WHERE clause, retrieve only the required columns instead of using SELECT *, and rewrite the query to make better use of available indexes."

9. Build an Executive KPI Query:

```
select orderstatus,  
count(*) as total_orders,  
round(sum(amount),2) as revenue,  
ROUND(AVG(amount),2) AS average_order_value  
from order_report  
group by orderstatus  
order by revenue desc;
```

Chapter 8 - Final Business Challenge (29 Questions)

1. What is the total revenue for the month?

```
with cte_rvnu as(  
  
select orderid, sum(amount) as total_rvnu  
  
from order_report  
  
group by orderid)  
  
select sum(total_rvnu)  
  
from cte_rvnu;
```

2. How many unique orders were received?

```
select count(distinct orderid)  
  
from order_report
```

3. How many unique customers placed orders?

```
select distinct customerid,  
  
concat(billfirstname,' ',billlastname) as full_name  
  
from order_report
```

4. What is the average order value?

```
WITH AVG_CTE AS (  
  
SELECT DISTINCT ORDERID, SUM(AMOUNT) AS TOTAL_AMT
```

```
FROM ORDER_REPORT
```

```
group by ORDERID)
```

```
SELECT ROUND(AVG(TOTAL_AMT),2) AS AVG_ORDER_VALUE
```

```
FROM AVG_CTE;
```

5. Which 10 customers generated the highest revenue?

```
select customerid,
```

```
concat(billfirstname,',billlastname) as full_name,
```

```
SUM(AMOUNT) AS TOTAL_AMT
```

```
from order_report
```

```
GROUP BY customerid, full_name
```

```
ORDER BY TOTAL_AMT DESC LIMIT 10;
```

6. Which 10 products generated the highest sales?

```
select uniqueid, item, itemdesc,
```

```
SUM(AMOUNT) AS TOTAL_AMT
```

```
from order_report
```

```
GROUP BY uniqueid, item, itemdesc
```

```
ORDER BY TOTAL_AMT DESC LIMIT 10;
```

7. Which products sold the highest quantity?

```
select uniqueid, item, itemdesc,
```

```
SUM(qty) AS TOTAL_QTY
```

```
from order_report
```

```
GROUP BY uniqueid, item, itemdesc
```

```
ORDER BY TOTAL_QTY DESC LIMIT 1;
```

8. Which states generated the highest revenue?

```
SELECT SHIPTOSTATE, SUM(AMOUNT) AS RVNU
```

```
FROM ORDER_REPORT
```

```
GROUP BY SHIPTOSTATE ORDER BY RVNU DESC LIMIT 1;
```

9. Which payment methods were used the most?

```

WITH CTE_PAYMENT AS (
SELECT DISTINCT ORDERID, paymenttype
FROM ORDER_REPORT)
SELECT paymenttype, COUNT(paymenttype) AS TOTAL_COUNT_TYPE
FROM CTE_PAYMENT
GROUP BY paymenttype ORDER BY TOTAL_COUNT_TYPE DESC

```

10. Which order statuses have the highest number of orders?

```

WITH CTE_STATUS AS (
SELECT DISTINCT ORDERID, ORDERSTATUS
FROM ORDER_REPORT)
SELECT ORDERSTATUS, COUNT(ORDERSTATUS) AS TOTAL_COUNT
FROM CTE_STATUS
GROUP BY ORDERSTATUS;

```

11. How much shipping revenue was collected?

```

select sum(shippingamount) as total_shipping
from (
select distinct orderid, shippingamount
from order_report) as subs;

```

12. How much sales tax was collected?

```

select sum(salestax) from order_report

```

13. How many orders used coupon codes?

```

select distinct orderid, couponcode
from order_report
where couponcode is not null;

```

14. Which coupon codes generated the most sales?

```

select couponcode, count(distinct orderid) as total_count,
sum(amount) as total_sales
from order_report

```


WHERE couponcode IS NOT NULL

group by couponcode order by total_count desc

15. How many loyalty customers placed orders?

select loyaltyname, count(distinct orderid) as total_count

from order_report

where loyaltyname <> 'Not Applicable'

group by loyaltyname

16. Which categories generated the highest revenue?

SELECT category, SUM(AMOUNT) AS RVNU

FROM ORDER_REPORT

GROUP BY category ORDER BY RVNU DESC;

17. Which main categories generated the highest revenue?

SELECT CATEGORYMAIN, SUM(AMOUNT) AS RVNU

FROM ORDER_REPORT

GROUP BY CATEGORYMAIN ORDER BY RVNU DESC ;

18. Which affiliates generated the highest sales?

SELECT affiliateorderno, paymenttype, SUM(AMOUNT) AS RVNU

FROM ORDER_REPORT

WHERE paymenttype = 'EBAY'

GROUP BY affiliateorderno, paymenttype

19. Which vendors received the most purchase orders?

SELECT VENDORNAME, COUNT(DISTINCT porderno) AS PO_COUNT,
SUM(totalcostprice)

FROM PURCHASE_ORDER

GROUP BY VENDORNAME ORDER BY PO_COUNT DESC;

20. Which products generated the highest gross profit?

SELECT productid, productcode, productname, totalsellingprice - totalcostprice
AS GROSS_RVNU

FROM PURCHASE_ORDER

```
GROUP BY productid, productcode, productname, GROSS_RVNU ORDER BY  
GROSS_RVNU DESC;
```

21. Which products had the lowest profit margin?

```
SELECT productid, productcode, productname, totalsellingprice - totalcostprice  
AS GROSS_RVNU  
  
FROM PURCHASE_ORDER  
  
GROUP BY productid, productcode, productname, GROSS_RVNU ORDER BY  
GROSS_RVNU ;
```

22. Compare total selling price vs total cost price by vendor.

```
SELECT VENDORNAME,  
  
       SUM(TOTALCOSTPRICE) As total_cost,  
  
       SUM(TOTALSELLINGPRICE) as total_sales,  
  
       sum(TOTALSELLINGPRICE - TOTALCOSTPRICE) as total_rvnu  
  
FROM PURCHASE_ORDER  
  
GROUP BY VENDORNAME order by total_rvnu desc;
```

23. Which purchase orders are still open?

```
select porderno, purchaseorderstatus  
  
from purchase_order  
  
where purchaseorderstatus = 'Open';
```

24. Which purchase orders have the highest value?

```
select porderno, sum(totalcostprice) as highest_value  
  
from purchase_order  
  
group by porderno order by highest_value desc limit 1;
```

25. Which products have the highest purchase quantity?

```
select productid, productcode, productname, uom, sum(qty) as total_qty  
  
from purchase_order  
  
group by productid, productcode, productname, uom order by total_qty desc  
limit 1;
```

26. Join customer orders with purchase orders and show order, vendor, customer, and product details.

```
select o.orderid,  
p.vendorname,  
o.customerid,  
concat (o.billfirstname, ',o.billlastname) as full_name,  
o.item,  
o.itemdesc  
from order_report o  
inner join purchase_order p  
on o.orderid = p.orderno
```

27. Create a monthly executive KPI summary (Revenue, Orders, Customers, Avg Order Value).

```
select date_trunc('month', orderdate)::date as month,  
sum(amount) as total_revenue,  
count(distinct orderid) as total_order_count,  
count(distinct customerid) as total_unique_customers,  
round(sum(amount) / count(distinct orderid),2) as avg_order_value  
from order_report  
group by DATE_TRUNC('month', orderdate)  
order by month;
```

28. Rank customers by revenue using Window Functions.

```
select customerid,  
concat (billfirstname, ',billlastname) as full_name,  
sum (amount) as total_customer_rvnu,  
rank () over (order by sum(amount) desc) as ranking  
from order_report  
group by customerid, full_name
```

29. Build a final business report combining sales, purchase cost, profit, shipping, tax, and vendor performance.

Ans:

```
with shipping_ctes as (  
  select  
    DATE_TRUNC('month', orderdate):: date as month,  
    sum (shippingamount) as total_shipping  
  from (  
    select distinct orderid,  
    orderdate,  
    shippingamount  
  from order_report) s  
  group by DATE_TRUNC('month', orderdate)  
) ,  
business_report as (  
  select date_trunc('month', o.orderdate):: date as month,  
    sum (o.amount) as total_sales,  
    sum (p.totalcostprice) as total_purchase_order_cost,  
    sum (o.amount - p.totalcostprice) as total_gross_pft,  
    sum(o.salestax) as total_sales_tax,  
    count (distinct p.vendorname) as active_vendors  
  from order_report o  
  inner join purchase_order p  
  on o.orderid = p.orderno  
  GROUP BY date_trunc('month', o.orderdate)  
)  
select b.month,  
       b.total_sales, b.total_purchase_order_cost, b.total_gross_pft,  
       ts.total_shipping,
```

```
        b.total_sales_tax,  
        b.active_vendors  
from business_report b  
left join shipping_ctes ts  
on b.month = ts.month  
order by b.month
```

Key Learning Outcomes :

- ✓ Sales Analysis
- ✓ Customer Analysis
- ✓ Product Analysis
- ✓ Financial Analysis
- ✓ Vendor Analysis
- ✓ Operations Analysis
- ✓ Joins
- ✓ Window Functions
- ✓ Executive Reporting

Skills Demonstrated

- ✓ SQL Basics
- ✓ Aggregate Functions
- ✓ String & Date Functions
- ✓ CASE & COALESCE
- ✓ Joins
- ✓ CTEs
- ✓ Subqueries
- ✓ Window Functions
- ✓ Views
- ✓ Indexes
- ✓ EXPLAIN ANALYZE
- ✓ Query Optimization
- ✓ Executive KPI Reporting
- ✓ Business Reporting
- ✓ Data Quality Checks